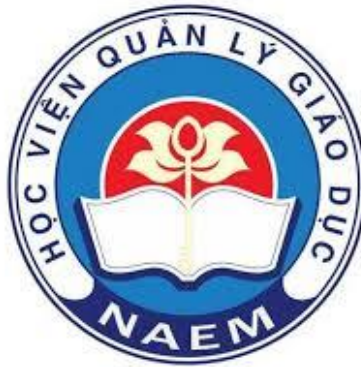


HỌC VIỆN QUẢN LÝ GIÁO DỤC  
KHOA CÔNG NGHỆ THÔNG TIN

\*\*\*\*\*



**BÁO CÁO**  
**KẾT THÚC HỌC PHẦN**  
**LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VỚI**  
**C++**  
**HỆ THỐNG BÁN HÀNG MINI**

Nhóm thực hiện: Nhóm 1

1. Nguyễn Minh Đức
2. Nguyễn Hồng Nam
3. Nguyễn Văn Huy

Giảng viên hướng dẫn: ThS. Ninh Thị Thanh Tâm

Hà Nội – 2026

# 1. MỤC LỤC

|       |                                              |    |
|-------|----------------------------------------------|----|
| 1.    | MỤC LỤC                                      | 2  |
| 2.    | DANH MỤC CÁC HÌNH VẼ                         | 5  |
| 3.    | DANH MỤC CÁC TỪ VIẾT TẮT                     | 6  |
| 1     | Lời mở đầu                                   | 7  |
| 1.1   | Giới thiệu về Đề tài                         | 7  |
| 1.2   | Mục tiêu nghiên cứu                          | 7  |
| 1.2.1 | Về mặt lý thuyết                             | 7  |
| 1.2.2 | Về mặt thực tiễn                             | 8  |
| 1.2.3 | Về kỹ năng khác                              | 8  |
| 1.3   | Đối tượng và Phạm vi nghiên cứu trong Đề tài | 8  |
| 1.3.1 | Đối tượng nghiên cứu                         | 8  |
| 1.3.2 | Phạm vi nghiên cứu                           | 8  |
| 1.4   | Phương pháp nghiên cứu                       | 9  |
| 1.4.1 | Phân tích và thiết kế hệ thống               | 9  |
| 1.4.2 | Lập trình thực nghiệm - Thử sai và sửa lỗi   | 9  |
| 1.4.3 | Làm việc nhóm – Quản lý Workflow/Mã nguồn    | 9  |
| 2     | Khảo sát, Đặc tả yêu cầu của Chương trình    | 9  |
| 2.1   | Mô tả yêu cầu bài toán                       | 9  |
| 2.1.1 | Quản lý phân cấp danh mục mặt hàng           | 10 |
| 2.1.2 | Quản lý dữ liệu bền vững dạng phi quan hệ    | 10 |
| 2.1.3 | Nghiệp vụ lập hóa đơn và kiểm soát giao dịch | 10 |
| 2.2   | Mô tả Use case chức năng chương trình        | 10 |
| 2.2.1 | Hiển thị danh sách sản phẩm                  | 10 |
| 2.2.2 | Thêm sản phẩm mới                            | 10 |
| 2.2.3 | In thông tin sản phẩm cụ thể                 | 10 |
| 2.2.4 | Xóa sản phẩm                                 | 11 |

|         |                                                                       |    |
|---------|-----------------------------------------------------------------------|----|
| 2.2.5   | Sửa thông tin sản phẩm                                                | 11 |
| 2.2.6   | Lập hóa đơn                                                           | 11 |
| 2.2.7   | In hóa đơn mới nhất                                                   | 11 |
| 2.2.8   | Xóa hóa đơn cũ nhất                                                   | 11 |
| 2.3     | Cấu trúc lưu trữ tệp CSDL phẳng (Flat file)                           | 11 |
| 2.3.1   | Tệp ( sanpham.txt )                                                   | 11 |
| 2.3.2   | Tệp ( hoadon.txt )                                                    | 11 |
| 3       | Phân tích chi tiết thiết kế Chương trình                              | 13 |
| 3.1     | Biểu đồ Lớp lõi (Core Class Diagram)                                  | 13 |
| 3.2     | Thiết kế chi tiết lớp lõi                                             | 15 |
| 3.2.1   | Lớp Cơ sở ( SanPham )                                                 | 15 |
| 3.2.2   | Lớp Dẫn xuất ( DoDienTu ) và ( DoGiaDung )                            | 18 |
| 3.2.3   | Lớp ( HoaDon ) và struct ( ChiTietHoaDon )                            | 19 |
| 3.2.3.1 | Struct ChiTietHoaDon                                                  | 19 |
| 3.2.3.2 | Lớp ( HoaDon )                                                        | 19 |
| 3.3     | Thiết kế chi tiết Hệ thống Điều hướng Giao diện Menu Console (Menu.h) | 21 |
| 3.3.1   | Quản lý Bộ Nhớ & Chuyển đổi dữ liệu File sang Đối tượng Object        | 21 |
| 3.3.2   | Các chức năng nghiệp vụ quản lý dữ liệu cơ bản (CRUD)                 | 22 |
| 3.3.3   | Luồng Nghiệp Vụ Bán Hàng & Hóa Đơn                                    | 22 |
| 3.3.4   | Hàm định dạng Giao diện Menu                                          | 23 |
| 3.4     | Chương trình chính kích hoạt hệ thống ( main.cpp )                    | 23 |
| 3.5     | Biểu đồ UML Use Case                                                  | 24 |
| 4       | Phân tích chi tiết Thuật toán Cốt lõi đã sử dụng                      | 25 |
| 4.1     | Thuật toán Tự động phát sinh Mã Sản phẩm không trùng lặp              | 25 |
| 4.1.1   | Nguyên lý thuật toán                                                  | 26 |

|       |                                                                                       |    |
|-------|---------------------------------------------------------------------------------------|----|
| 4.1.2 | Thuật toán phục hồi trạng thái bộ đếm từ File khi khởi động                           | 26 |
| 4.2   | Thuật toán Chuẩn hóa Dữ liệu Giá bán (Data Normalization)                             | 27 |
| 4.3   | Thuật toán Đọc/Ghi và Phân tích Cú pháp Bản ghi phẳng (Flat-file Parsing) của SanPham | 30 |
| 4.4   | Thuật toán Đọc/Ghi và Lưu trữ Hóa đơn                                                 | 32 |
| 4.4.1 | Thuật toán Lưu hóa đơn xuống file ( HoaDon::LuuFile )                                 | 32 |
| 4.4.2 | Thuật toán Đồng bộ Đọc/Ghi Hệ thống                                                   | 35 |
| 4.4.3 | Thuật toán Đọc và Xử lý nâng cao dữ liệu Hóa đơn                                      | 38 |
| 5     | Kết quả thực nghiệm – Demo chương trình                                               | 41 |
| 5.1   | Môi trường thực nghiệm                                                                | 41 |
| 5.2   | Kịch bản kiểm thử và Demo chức năng hệ thống                                          | 41 |
| 5.2.1 | Giao diện Menu chính khi khởi động chương trình                                       | 42 |
| 5.2.2 | Chức năng Hiển thị danh sách và Chuẩn hóa dữ liệu (Chức năng 2)                       | 43 |
| 5.2.3 | Chức năng Lập hóa đơn và Đồng bộ hóa kho hàng (Chức năng 6)                           | 43 |
| 6     | KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN                                                          | 46 |
| 7     | DANH MỤC TÀI LIỆU THAM KHẢO                                                           | 48 |
| 8     | PHÂN CÔNG CÔNG VIỆC NHÓM                                                              | 50 |

## **2. DANH MỤC CÁC HÌNH VẼ**

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Hình 1: Biểu đồ Lớp lỗi – Bản vẽ tay prototype ban đầu.....                  | 13 |
| Hình 2: Biểu đồ Lớp lỗi – Bản vẽ UML Diagram hoàn chỉnh.....                 | 14 |
| Hình 3: Biểu đồ UML Use Case.....                                            | 23 |
| Hình 4: Giao diện Menu chính khi khởi động chương trình.....                 | 38 |
| Hình 5: Chức năng Hiển thị danh sách và Chuẩn hóa dữ liệu (Chức năng 2)..... | 39 |
| Hình 6: Chức năng Lập hóa đơn và Đồng bộ hóa kho hàng (Chức năng 6) (1)..... | 40 |
| Hình 7: Chức năng Lập hóa đơn và Đồng bộ hóa kho hàng (Chức năng 6) (2)..... | 40 |
| Hình 8: Chức năng Lập hóa đơn và Đồng bộ hóa kho hàng (Chức năng 6) (3)..... | 41 |

### 3. DANH MỤC CÁC TỪ VIẾT TẮT

| Từ viết tắt                    | Giải nghĩa                                                               |
|--------------------------------|--------------------------------------------------------------------------|
| OOP                            | Object-Oriented Programming – Lập trình hướng đối tượng                  |
| NNLT                           | Ngôn ngữ lập trình                                                       |
| I/O                            | Input/Output – Vào/Ra                                                    |
| RDBMS                          | Relational Database Management System – Hệ quản trị CSDL quan hệ         |
| CSDL                           | Cơ sở dữ liệu                                                            |
| DBMS                           | Database Management System – Hệ quản trị CSDL                            |
| Console /<br>Console Interface | Giao diện tương tác dòng lệnh/ Giao diện dòng lệnh/ Môi trường dòng lệnh |
| GUI                            | Graphical User Interface – Giao diện đồ họa                              |
| IDE                            | Integrated Development Environment                                       |
| CRUD                           | Create – Read – Update – Delete<br>Tạo – Đọc – Cập nhật – Xóa            |
| UML                            | <b>Unified Modeling Language</b>                                         |
|                                |                                                                          |

# 1 Lời mở đầu

## 1.1 Giới thiệu về Đề tài

Trong kỷ nguyên số hóa và chuyển đổi số đang diễn ra mạnh mẽ ngày nay, việc ứng dụng công nghệ thông tin vào quản lý hoạt động kinh doanh đã trở thành một xu hướng tất yếu và là điều kiện tiên quyết để nâng cao năng lực cạnh tranh của các doanh nghiệp. Đối với các mô hình kinh doanh bán lẻ như cửa hàng tiện lợi, siêu thị mini, việc quản lý thủ công bằng sổ sách hay các công cụ văn phòng thô sơ như Excel đã không còn đáp ứng được yêu cầu về tốc độ, tính chính xác và khả năng lưu trữ dữ liệu lâu dài. Quản lý thủ công thường dẫn đến các sai sót trong tính toán hóa đơn, nhầm lẫn thông tin giữa các danh mục sản phẩm khác nhau (đặc biệt là giữa nhóm hàng điện tử và gia dụng có các thuộc tính đặc thù riêng), đồng thời gây khó khăn cho việc thống kê doanh thu theo thời gian thực.

Xuất phát từ thực tế đó, việc xây dựng một phần mềm quản lý là vô cùng cấp bách. Trong khuôn khổ học phần **Lập trình hướng đối tượng C++**, nhóm chúng em đã lựa chọn đề tài: **"Xây dựng Hệ thống bán hàng mini"**. Đề tài này không chỉ dừng lại ở việc giải quyết bài toán quản lý bán hàng thực tế trên giao diện Console mà còn là cơ hội để nhóm hiện thực hóa, kiểm chứng các lý thuyết cốt lõi của phương pháp luận hướng đối tượng – một tư duy lập trình nền tảng và quan trọng bậc nhất đối với một kỹ sư phần mềm tương lai.

Nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc tới cô giáo ThS. Ninh Thị Thanh Tâm, người đã định hướng và đưa ra những chỉ dẫn học thuật quý báu trong suốt quá trình nhóm triển khai đề tài này. Do giới hạn về mặt thời gian, hệ thống chắc chắn không tránh khỏi những thiếu sót, nhóm rất mong nhận được những đóng góp từ cô và các bạn.

## 1.2 Mục tiêu nghiên cứu

Đề tài được thực hiện nhằm hướng tới các mục tiêu cụ thể sau:

### 1.2.1 Về mặt lý thuyết

Đề tài giúp hiểu sâu sắc và áp dụng thành thạo các đặc tính nền tảng của lập trình hướng đối tượng bao gồm: Tính đóng gói (Encapsulation) để bảo vệ dữ liệu sản phẩm;

tính kế thừa (Inheritance) để tối ưu hóa mã nguồn khi phân loại hàng hóa; tính đa hình (Polymorphism) và kỹ thuật nạp chồng toán tử (Operator Overloading) để tối ưu hóa các thao tác xử lý nghiệp vụ toán học trên các đối tượng phức tạp như Hóa đơn.

### **1.2.2 Về mặt thực tiễn**

Đề tài yêu cầu xây dựng thành công một ứng dụng phần mềm có khả năng chạy ổn định, giao diện Console trực quan cho phép người sử dụng dễ dàng quản lý kho hàng (thêm, sửa, xóa sản phẩm điện tử, gia dụng), tự động hóa quy trình lập hóa đơn, tính tiền chính xác và có cơ chế lưu trữ dữ liệu an toàn, đảm bảo tính toàn vẹn của thông tin dưới dạng file để phục vụ cho các lần sử dụng kế tiếp.

### **1.2.3 Về kỹ năng khác**

Đề tài giúp hoàn thiện quy trình làm việc nhóm chuyên nghiệp, biết cách phân chia module bài toán độc lập và phối hợp quản lý mã nguồn đồng bộ thông qua hệ thống kiểm soát phiên bản Git và nền tảng lưu trữ mã nguồn GitHub.

## ***1.3 Đối tượng và Phạm vi nghiên cứu trong Đề tài***

### **1.3.1 Đối tượng nghiên cứu**

Các đối tượng thực tế trong một cửa hàng bán lẻ bao gồm: Thông tin hàng hóa tổng quát (Hàng hóa/Sản phẩm), các ngành hàng chuyên biệt có thuộc tính riêng (Đồ điện tử, Đồ gia dụng), và thực thể trung tâm của quá trình giao dịch (Hóa đơn/Hóa đơn bán hàng).

### **1.3.2 Phạm vi nghiên cứu**

- Về mặt công nghệ: Phần mềm được viết thuần túy bằng ngôn ngữ lập trình (NNLT) C++ (chuẩn NNLT C++ 11 trở lên). Phần mềm sử dụng các luồng/thư viện I/O tiêu chuẩn (iostream)
- Về mặt dữ liệu: Chương trình sẽ dùng hệ thống quản lý lưu trữ dưới dạng các file văn bản thô (.txt) để mô phỏng một hệ cơ sở dữ liệu thu nhỏ, chưa triển khai và kết nối sử dụng tới các Hệ quản trị cơ sở dữ liệu quan hệ phức tạp.



- Về mặt giao diện phần mềm: Phần mềm hoạt động trên Console Interface, tập trung tối đa vào cấu trúc xử lý logic thay vì sử dụng giao diện đồ họa (GUI).

## ***1.4 Phương pháp nghiên cứu***

Đề hoàn thành dự án Đề tài này, nhóm chúng em đã kết hợp các phương pháp nghiên cứu sau:

### **1.4.1 Phân tích và thiết kế hệ thống**

Tiến hành khảo sát quy trình bán hàng thực tế tại các siêu thị mini, từ đó bóc tách các thực thể, xác định các thuộc tính và hành vi của từng đối tượng nhằm xây dựng sơ đồ lớp (Class Diagram) chuẩn xác.

### **1.4.2 Lập trình thực nghiệm - Thử sai và sửa lỗi**

Sử dụng các phần mềm môi trường phát triển tích hợp (IDE) có trình biên dịch sẵn (Compiler) phổ biến như Code::Blocks, Visual Studio để trực tiếp viết và cài đặt thuật toán, kiểm thử (Testing) từng module chức năng, phát hiện lỗi (bug), sửa bug và tối ưu hóa hiệu năng mã nguồn.

### **1.4.3 Làm việc nhóm – Quản lý Workflow/Mã nguồn**

Áp dụng mô hình phân rã bài toán, mỗi thành viên chịu trách nhiệm cài đặt các lớp độc lập, sau đó tiến hành tích hợp lại thành một và kiểm tra xung đột code luân phiên thông qua công cụ đồ họa GitHub Desktop.

## **2 Khảo sát, Đặc tả yêu cầu của Chương trình**

### ***2.1 Mô tả yêu cầu bài toán***

Hệ thống bán hàng mini yêu cầu quản lý hai đối tượng cốt lõi: Sản phẩm trong kho và Hóa đơn bán hàng. Hệ thống yêu cầu số hóa toàn bộ quy trình vận hành luân chuyển hàng hóa và quản lý tài chính hóa đơn trong một cửa hàng bán lẻ. Cụ thể bao gồm các nghiệp vụ cốt lõi sau:

### **2.1.1 Quản lý phân cấp danh mục mặt hàng**

Hệ thống cần theo dõi hai nhóm hàng có tính chất nghiệp vụ khác nhau là Đồ điện tử (DDT) và Đồ gia dụng (DGD). Mọi mặt hàng đều chia sẻ thuộc tính chung như tên, nhà sản xuất, đơn giá và số lượng tồn, nhưng cần cơ chế định danh để phân biệt.

### **2.1.2 Quản lý dữ liệu bền vững dạng phi quan hệ**

Dữ liệu sản phẩm và hóa đơn phải duy trì liên tục qua các phiên làm việc. Phần mềm phải đọc và phân tích (parse) tự động luồng văn bản từ tệp tin, cho phép thêm mới, cập nhật thông tin và loại bỏ sản phẩm trực tiếp trên file tĩnh.

### **2.1.3 Nghiệp vụ lập hóa đơn và kiểm soát giao dịch**

Người dùng có thể tạo hóa đơn thanh toán cho khách hàng bằng cách thêm các mặt hàng với số lượng mong muốn. Quá trình này bắt buộc phải kiểm tra tính toàn vẹn dữ liệu: nếu số lượng mua vượt quá số lượng hiện có trong kho, hệ thống phải chặn giao dịch. Khi hóa đơn hoàn thành, hệ thống tính tổng tiền, giảm trừ số lượng tồn kho thực tế, xuất hóa đơn chi tiết trực quan và lưu vết không ghi đè vào lịch sử hệ thống.

## **2.2 Mô tả Use case chức năng chương trình**

Hệ thống cung cấp một Menu tương tác gồm 8 chức năng chính dành cho Người quản lý/Người dùng:

### **2.2.1 Hiện thị danh sách sản phẩm**

Đọc từ file và in ra màn hình tất cả các sản phẩm dạng chuẩn hóa.

### **2.2.2 Thêm sản phẩm mới**

Cho phép chọn loại sản phẩm (DDT/DGD), nhập thông tin và tự động cấp mã định danh.

### **2.2.3 In thông tin sản phẩm cụ thể**

Định vị sản phẩm bằng mã sản phẩm và in thông tin chi tiết về sản phẩm đó.

#### 2.2.4 Xóa sản phẩm

Loại bỏ hoàn toàn một mặt hàng khỏi kho dữ liệu tính qua mã sản phẩm.

#### 2.2.5 Sửa thông tin sản phẩm

Cập nhật lại giá bán hoặc số lượng khi nhập thêm hàng hay muốn thay đổi thông tin của mặt hàng. Xác định mặt hàng cần sửa bằng mã sản phẩm.

#### 2.2.6 Lập hóa đơn

Đây là nghiệp vụ cốt lõi, có cộng dồn số lượng khi bán nhiều sản phẩm, tự động cập nhật giảm kho và tính tiền.

#### 2.2.7 In hóa đơn mới nhất

Truy xuất giao dịch vừa thực hiện gần nhất để kiểm tra.

#### 2.2.8 Xóa hóa đơn cũ nhất

Cơ chế giải phóng tài nguyên dữ liệu lịch sử thủ công khi đã xác định không còn dùng đến nhằm tránh phình to CSDL gây giảm hiệu suất xử lý.

### 2.3 Cấu trúc lưu trữ tệp CSDL phẳng (Flat file)

Để duy trì dữ liệu bền vững (Persistence) mà không cần hệ quản trị CSDL phức tạp, hệ thống tổ chức dữ liệu dưới dạng các tệp văn bản (.txt) với quy ước phân tách trường bằng ký tự gạch đứng (|).

#### 2.3.1 Tệp ( sanpham.txt )

Mỗi dòng là một bản ghi sản phẩm:

- *Cấu trúc:* LoaiSP|MaSP|TenSP|NhaSanXuat|GiaBan|SoLuongTon
- *Ví dụ thực tế:* DDT|XM01|Dien thoai Redmi Note 15 5G|Xiaomi|7000000|700

#### 2.3.2 Tệp ( hoadon.txt )

Lưu trữ lịch sử biên lai dạng báo cáo trực quan phục vụ lưu vết kế toán, phân tách các giao dịch bằng chuỗi ký tự biên =====.

- *Cấu trúc đầu:* HOA DON BAN HANG: Ma HD

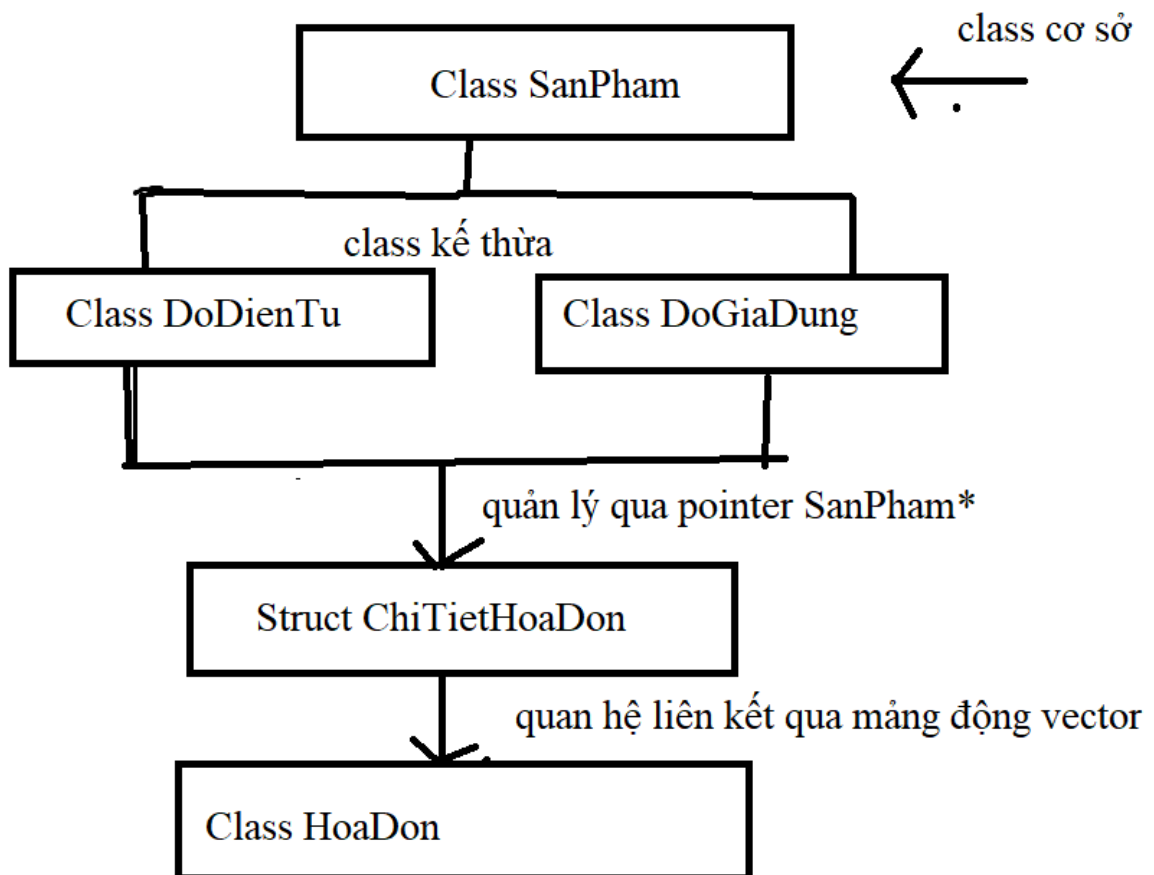
- Dòng chi tiết: STT. MaSP: [Mã] | Ten: [Tên] | NSX: [Hãng] | Đơn giá: [Giá]  
VND | SL: [Số lượng] | Thanh tiền: [Tiền] VND
- Cấu trúc cuối: TONG TIEN: [Tiền] VND

### 3 Phân tích chi tiết thiết kế Chương trình

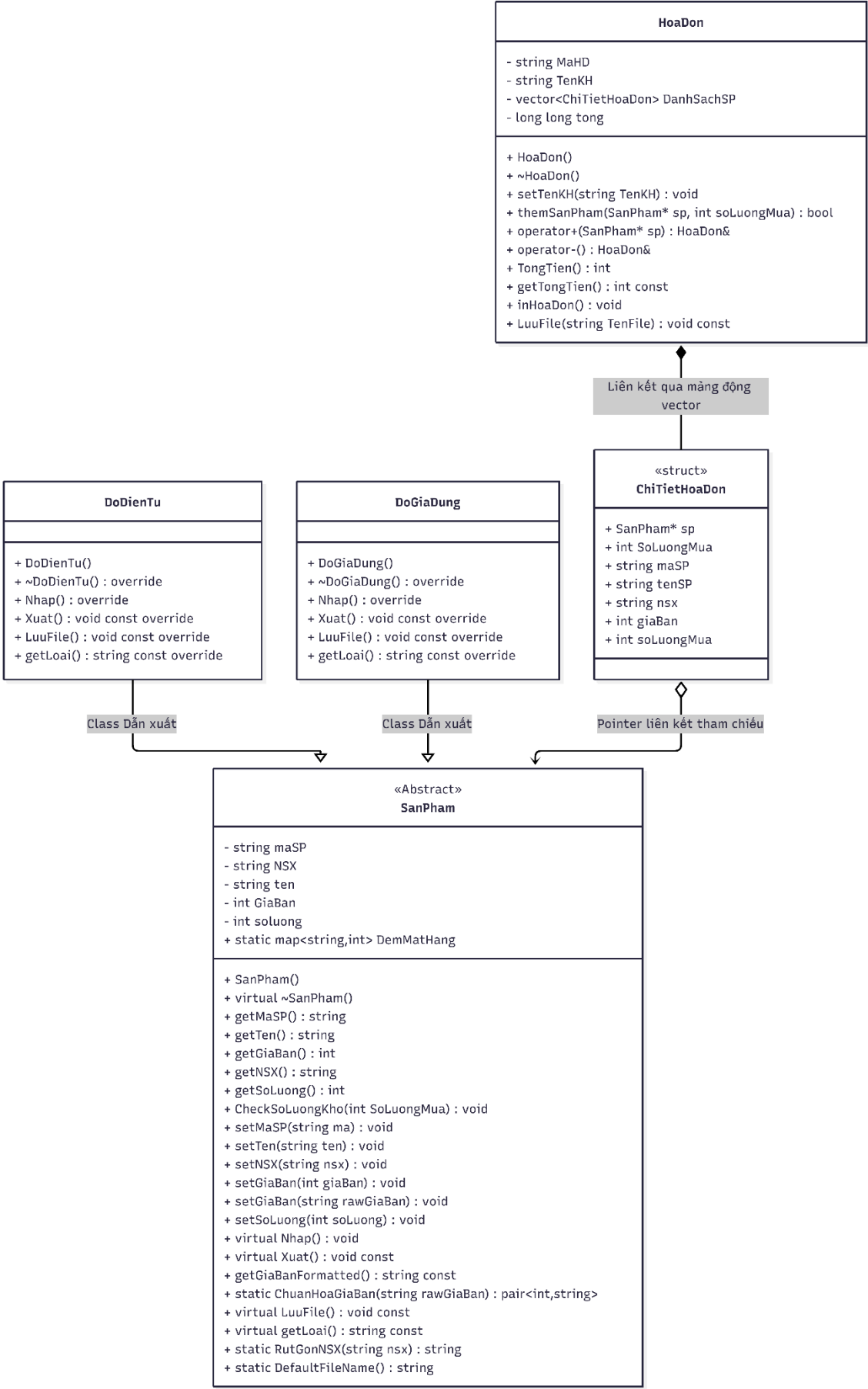
#### 3.1 Biểu đồ Lớp lõi (Core Class Diagram)

Các lớp lõi là các lớp chủ lực, là bộ não của chương trình. Hệ thống tổ chức theo kiến trúc hướng đối tượng chặt chẽ: DoDienTu và DoGiaDung kế thừa công khai (public inheritance) từ lớp cơ sở SanPham. Lớp HoaDon liên kết với lớp SanPham thông qua cấu trúc thành phần ChiTietHoaDon quản lý bằng mảng động std::vector.

Biểu đồ vẽ tay ban đầu:



Biểu đồ UML hoàn chỉnh sau khi phát triển phần mềm:



## 3.2 *Thiết kế chi tiết lớp lỗi*

### 3.2.1 Lớp Cơ sở ( SanPham )

```
class SanPham
{
    private:
        string maSP, NSX, ten;
        int GiaBan;
        int soluong;
    public:
        SanPham();
        //tranh ro ri bo nho
        virtual ~SanPham();

        //dung de cac class khac (HoaDon, KhachHang)
        lay du lieu
        string getMaSP() const;

        string getTen() const;
        int getGiaBan() const;
        string getNSX() const;

        //Khach hang mua bao nhieu
        int getSoLuong() const {return soluong;}

        //check soluong trong kho
        void CheckSoLuongKho(int SoLuongMua) {soluong
        -= SoLuongMua;}
```

```

    void setMaSP(const string &ma);
    void setTen(const string &ten);
    void setNSX(const string &nsx);
    // Gán giá bán từ giá trị số nguyên cho mục
    đích tính toán
    void setGiaBan(int giaBan);

    // Dùng khi nhập giá từ chuỗi có thể có dấu
    "." hoặc ký tự khác
    void setGiaBan(const string &rawGiaBan);

    void setSoLuong(int soLuong);

    virtual void Nhap();
    virtual void Xuat() const;

    // Trả về chuỗi giá có định dạng dấu chấm để
    in ra màn hình
    string getGiaBanFormatted() const;

    // Chuẩn hóa đầu vào giá tiền
    static pair<int, string> ChuanHoaGiaBan(const
    string &rawGiaBan);

    virtual void LuuFile() const = 0;
    virtual string getLoai() const = 0;
    static string RutGonNSX(const string &nsx);
    static const string &DefaultFileName();
    static map<string, int> DemMatHang;
};

```



- Thuộc tính **protected/private**: string maSP, NSX, ten; int GiaBan, soluong;

- Phương thức:

- **virtual void Nhap(), virtual void Xuat() const**: Hàm ảo nhập xuất luồng dữ liệu.
- **virtual void LuuFile() const = 0, virtual string getLoai() const = 0**: Các hàm thuần ảo (Pure virtual functions) định hình cấu trúc lớp trừu tượng (Abstract class).
- Constructor **SanPham()** khởi tạo một đối tượng sản phẩm mặc định với các chuỗi rỗng (“”) và giá bán, số lượng = 0.
- Virtual destructor **~SanPham()**: Hàm hủy ảo. Việc thêm từ khóa **virtual** rất quan trọng vì nó đảm bảo khi giải phóng một con trỏ lớp cha **SanPham\*** đang trỏ tới lớp con (**DoDienTu/DoGiaDung**), hàm hủy của lớp con sẽ được gọi trước, tránh hiện tượng rò rỉ bộ nhớ (memory leak).
- Các hàm Getter (**getMaSP, getTen, getGiaBan, getNSX, getSoLuong**): Trả về giá trị của các thuộc tính private tương ứng (Đảm bảo tính đóng gói).
- Các hàm Setter (**setMaSP, setTen, setNSX, setGiaBan, setSoLuong**): Gán giá trị mới cho thuộc tính của sản phẩm. Riêng hàm **setGiaBan(const string &rawGiaBan)** sẽ tự động gọi hàm chuẩn hóa để lọc bỏ các ký tự lạ.
- **CheckSoLuongKho(int SoLuongMua)**: Giảm số lượng tồn kho đi một lượng bằng SoLuongMua khi có khách hàng chọn mua mặt hàng này.
- **static pair<int,string> ChuanHoaGiaBan(const string &raw)**: Hàm tĩnh xử lý định dạng chuỗi.
- **static string RutGonNSX(const string &nsx)**: Rút gọn tên hãng nhà sản xuất với 1 quy luật cố định nhằm dễ dàng như trũ.
- **getGiaBanFormatted()**: Trả về giá tiền với định dạng (format) chuẩn dễ nhìn (có các dấu chấm ngăn cách cấp nghìn, trăm nghìn, triệu, tỷ,...)

### 3.2.2 Lớp Dẫn xuất ( DoDienTu ) và ( DoGiaDung )

```
class DoDienTu : public SanPham
{
    public:
        DoDienTu();
        //override: ghi đè lên class SanPham, giúp
        compiler kiểm tra thông tin
        ~DoDienTu() override;
        void Nhap() override;
        void Xuat() const override;
        void LuuFile() const override;
        string getLoai() const override;
};

class DoGiaDung : public SanPham
{
    public:
        DoGiaDung();
        ~DoGiaDung() override;
        void Nhap() override;
        void Xuat() const override;
        void LuuFile() const override;
        string getLoai() const override;
};
```

Ghi đè (override) toàn bộ nhóm hàm thuần ảo của lớp cha nhằm trả về định danh cụ thể: getLoai() trả về "DDT" hoặc "DGD", và cụ thể hóa phương thức ghi tệp tĩnh của riêng chúng.

### 3.2.3 Lớp ( HoaDon ) và struct ( ChiTietHoaDon )

#### 3.2.3.1 Struct ChiTietHoaDon

Là một cấu trúc lưu trữ thông tin của **một dòng hàng** trong hóa đơn, gồm con trỏ quản lý sản phẩm gốc, số lượng đặt mua, kèm theo việc sao chép lại các thông tin thời điểm mua (mã, tên, nhà sản xuất, giá bán) nhằm lưu vết hóa đơn độc lập.

```
struct ChiTietHoaDon
{
    SanPham *sp = nullptr;
    int SoLuongMua = 0;
    string maSP;
    string tenSP;
    string nsx;
    int giaBan = 0;
    int soLuongMua = 0;
};
```

#### 3.2.3.2 Lớp ( HoaDon )

```
class HoaDon
{
    private:
        string MaHD;
        string TenKH;
        vector <ChiTietHoaDon> DanhSachSP;
        long long tong = 0;
    public:
        HoaDon() ;
```

```

    ~HoaDon();

    int TongTien();

    void inHoaDon(); //const;

    bool themSanPham(SanPham* sp, int
soLuongMua);

    HoaDon& operator+ (SanPham* sp); //cong don hoa
don

    HoaDon& operator- (SanPham* sp); //xoa sp truoc
khi khach hang doi y

    void setTenKH(const string &TenKH);

    int getTongTien() const;

    void LuuFile(const string &tenFile) const;

};

```

- Constructor **HoaDon()**: Khởi tạo hóa đơn mặc định với các thông số và logic cài đặt định sẵn: Tự động sinh mã hóa đơn ngẫu nhiên dưới dạng chuỗi kí tự (ví dụ: Ma HD 1234) nhờ hàm **rand()** và đặt tên khách hàng mặc định thành rỗng.
- Destructor **~HoaDon()**: Giải phóng bộ nhớ bằng cách xóa sạch danh sách các mặt hàng đã chọn mua nằm trong **vector**
- **setTenKH(const string &TenKH)**: Cập nhật tên của khách hàng mua đơn hàng này.
- **themSanPham(SanPham\* sp, int soLuongMua)**: Kiểm tra tính hợp lệ (sản phẩm tồn tại không, số lượng mua có > 0 không). Sau đó đối chiếu với số lượng tồn kho của mặt hàng. Nếu kho đủ đáp ứng, nó gọi **sp->CheckSoLuongKho(soLuongMua)** để trừ bớt kho gốc, đóng gói thông tin vào cấu trúc **ChiTietHoaDon** rồi đẩy (push\_back) vào danh sách mặt hàng của hóa đơn.
- **operator+(SanPham\* sp)**: Quá tải toán tử cộng (operator overloading). Cho phép thêm nhanh mặt hàng vào hóa đơn bằng cú pháp trực quan: **hoaDon + sanPham;**. Hàm sẽ hỏi số lượng mua từ bàn phím trực tiếp rồi gọi

**themSanPham()**. Cuối cùng trả về **\*this** để có thể cộng dồn liên tiếp nhiều sản phẩm trên một dòng lệnh.

- **operator-():** Quá tải toán tử trừ. Được sử dụng như một nút "Undo" (Hủy tác vụ). Khi khách hàng đổi ý, toán tử này sẽ lấy dòng sản phẩm cuối cùng ra khỏi DanhSachSP (DanhSachSP.pop\_back()), tính toán lại số lượng mua và cộng hoàn trả ngược lại kho RAM (setSoLuong(getSoLuong() + soLuongMua)).
- **getTongTien() const và TongTien():** Duyệt qua toàn bộ danh sách **ChiTietHoaDon** đang có, tính **giaBan \* soLuongMua** của từng món rồi cộng dồn lại để trả về tổng giá trị hóa đơn.
- **inHoaDon():** Xuất phiếu tính tiền trực quan ra màn hình Console, bao gồm: Mã HD, Tên KH, bảng danh sách sản phẩm cấu trúc dạng dòng, đơn giá, thành tiền từng loại và tổng tiền thanh toán sau cùng.
- **LuuFile(const string &TenFile) (Const):** Mở file hóa đơn (hoadon.txt) ở chế độ ghi tiếp (ios::app) để append thêm biên lai mới vào cuối file mà không ghi đè mất lịch sử các hóa đơn cũ.

### 3.3 *Thiết kế chi tiết Hệ thống Điều hướng Giao diện Menu Console (Menu.h)*

File này đóng vai trò như bộ điều khiển trung tâm (Controller), kết nối tầng giao diện người dùng với tầng dữ liệu file.

#### 3.3.1 **Quản lý Bộ Nhớ & Chuyển đổi dữ liệu File sang Đối tượng Object**

- **giaiPhongDanhSachSanPham(vector<SanPham\*>& danhSach):** Duyệt qua vector con trỏ, thực hiện delete từng vùng nhớ đối tượng sản phẩm được cấp phát động, sau đó .clear() danh sách để tránh lọt con trỏ rác.
- **taoSanPhamTuDong(...):** Đóng vai trò như một *Factory Method đơn giản*. Dựa vào chuỗi định danh "DDT" hay "DGD" đọc từ file, hàm sẽ dùng từ khóa new để khởi tạo chính xác vùng nhớ cho DoDienTu hoặc DoGiaDung. Đồng thời cập nhật lại bộ đếm tĩnh **SanPham::DemMatHang** để đảm bảo khi người dùng tạo sản phẩm mới tiếp theo, số thứ tự ID không bị trùng lặp với dữ liệu cũ trong file.
- **docTatCaSanPham(const string& tenFile):** Mở file dữ liệu, đọc từng dòng văn bản, tách các thuộc tính qua phân cách ( | ) bằng stringstream, chuyển đổi kiểu

dữ liệu tương ứng rồi gọi `taoSanPhamTuDong` để nạp toàn bộ danh sách sản phẩm vào một **vector<SanPham\*>** trên RAM.

### 3.3.2 Các chức năng nghiệp vụ quản lý dữ liệu cơ bản (CRUD)

- **timSanPhamTheoMa(...):** Duyệt vòng lặp trong vector để tìm kiếm sản phẩm có mã trùng khớp.
- **inSanPhamTheoMa(...):** Đọc dữ liệu từ file lên RAM, tìm sản phẩm và gọi hàm `Xuat()` của nó, sau đó giải phóng bộ nhớ ngay lập tức.
- **luuDanhSachSanPham(...):** Ghi đè lại toàn bộ nội dung file (`ios::trunc`) bằng dữ liệu mới nhất từ vector trên RAM. Thường dùng sau khi Xóa hoặc Sửa đổi thông tin.
- **inTatCaSanPhamTuFile(...):** Đọc file, duyệt danh sách và in toàn bộ sản phẩm hiện có ra màn hình.
- **xoaSanPham(...):** Tìm sản phẩm theo mã trong danh sách RAM, tiến hành xóa phần tử, giải phóng vùng nhớ của nó rồi lưu đè lại file.
- **suaSanPham(...):** Tìm sản phẩm, hiển thị thông tin cũ, cho phép người dùng nhấn Enter để bỏ qua hoặc gõ giá trị mới (Tên, NSX, Giá, Kho) để cập nhật, sau đó cập nhật lại file.

### 3.3.3 Luồng Nghiệp Vụ Bán Hàng & Hóa Đơn

- **lapHoaDonBanHang(...):** Đây là hàm phức hợp quan trọng nhất:
  - Tải danh sách sản phẩm từ file lên bộ nhớ tạm (RAM).
  - Khởi tạo một đối tượng `HoaDon`.
  - Mở vòng lặp cho phép thu ngân nhập liên tiếp các mã sản phẩm và số lượng cần mua.
  - Cập nhật trực tiếp số lượng tồn kho trên RAM thông qua đối tượng hóa đơn.
  - Kết thúc mua hàng, hàm tiến hành in hóa đơn ra màn hình, ghi dữ liệu biên lai vào file ( **hoadon.txt** ) và đồng thời cập nhật số lượng tồn kho mới vào file ( **sanpham.txt** ).
- **docTatCaHoaDon(...):** Đọc file văn bản ( **hoadon.txt** ), nhận diện các khối văn bản nằm giữa các đường kẻ ( ===== )

để phân tách và đóng gói từng hóa đơn thành một chuỗi **string** riêng biệt, trả về một danh sách các chuỗi hóa đơn.

- **inHoaDonMoiNhatTuFile(...)**: Lấy phần tử cuối cùng ( **.back()** ) từ danh sách chuỗi hóa đơn vừa đọc để hiển thị ra màn hình.
- **xoaHoaDonCuNhat(...)**: Xóa phần tử đầu tiên ( **.begin()** ) trong mảng chuỗi hóa đơn (tương ứng hóa đơn xuất hiện sớm nhất ở đầu file) rồi lưu các hóa đơn còn lại đè vào file.

### 3.3.4 Hàm định dạng Giao diện Menu

- **canGiua(...), inDuongKeMenu(...), inDongCanGiuaMenu(...), inDongMenu(...)**: Các hàm hỗ trợ xử lý chuỗi và độ rộng **std::setw** để căn lề giữa, vẽ các góc ( +, |, - ) giúp khung menu hiển thị đẹp, vuông vắn trên màn hình console.
- **hienThiMenu()**: In toàn bộ bảng danh sách 9 lựa chọn chức năng ra màn hình.
- **xuLyLuaChon(int luaChon)**: Cấu trúc **switch-case** tương ứng với lựa chọn từ 1 đến 9 để kích hoạt các hàm xử lý logic đã phân tích ở trên.

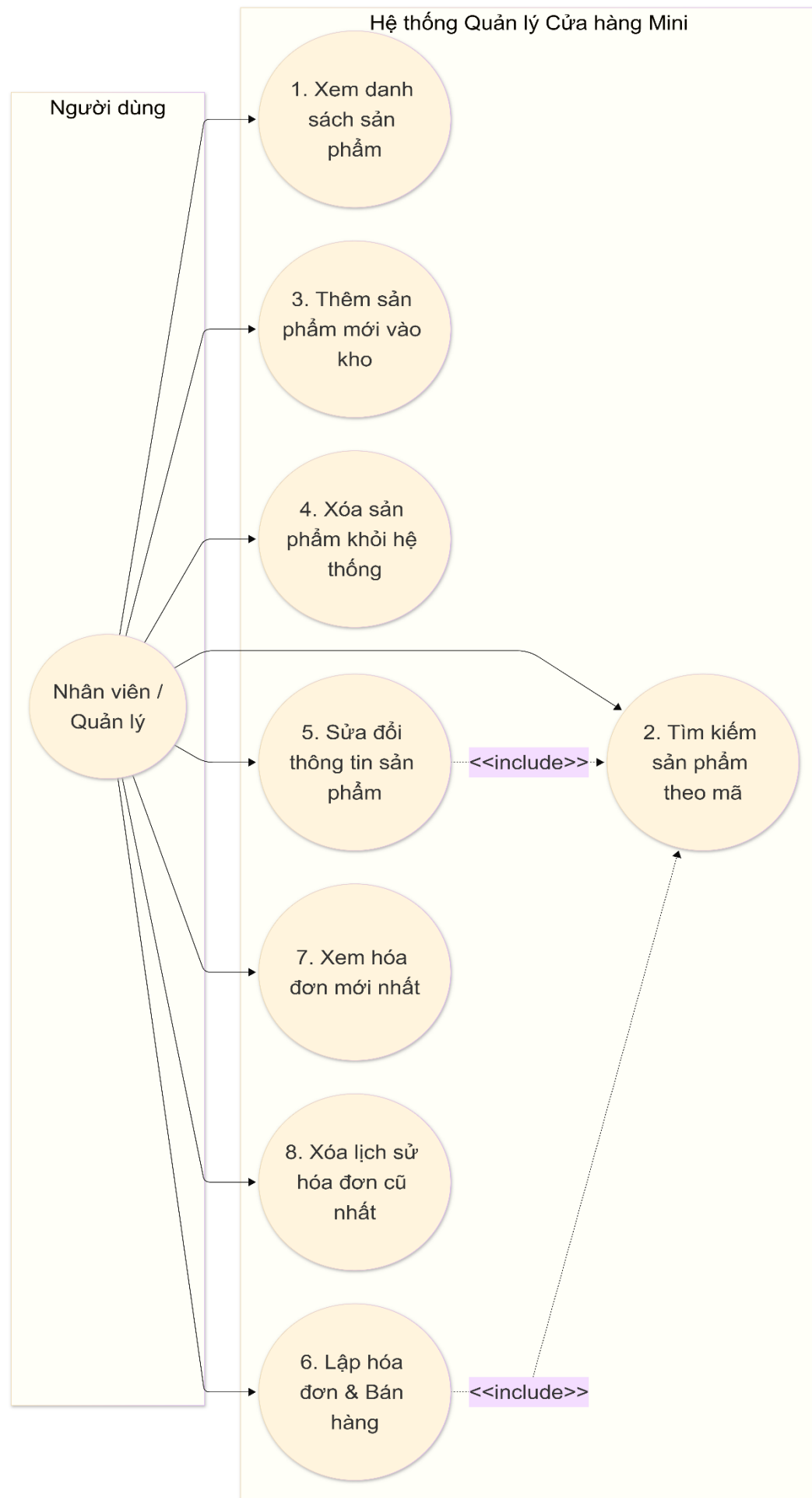
## 3.4 Chương trình chính kích hoạt hệ thống ( *main.cpp* )

- **map<string,int> SanPham::DemMatHang::** Định nghĩa biến tĩnh toàn cục cho lớp **SanPham** để cấp phát bộ nhớ cho biến đếm STT sản phẩm.
- **main()**: Chứa một vòng lặp vô hạn **while (true)**. Trong mỗi chu kỳ, chương trình sẽ dựng giao diện qua **hienThiMenu()**, bắt lỗi nhập liệu từ người dùng (nếu nhập chữ thay vì nhập số sẽ tự động xóa bộ đệm lỗi bằng **cin.clear()** và **cin.ignore()**). Khi người dùng chọn 9, lệnh **break** được kích hoạt để kết thúc và đóng chương trình một cách an toàn.

### 3.5 Biểu đồ UML

#### Use Case

Mô tả toàn bộ các chức năng nghiệp vụ mà người vận hành hệ thống (Nhân viên / Quản lý) có thể thực hiện thông qua giao diện Menu điều khiển.





## 4 Phân tích chi tiết Thuật toán Cốt lõi đã sử dụng

### 4.1 Thuật toán Tự động phát sinh Mã Sản phẩm không trùng lặp

Để giải phóng người dùng khỏi việc quản lý mã, hệ thống sử dụng cơ chế sinh mã tự động dựa trên tiền tố nhà sản xuất và bộ đếm tính.

```
string SanPham::RutGonNSX(const string &nsx)
{
    if (nsx.length() <= 2)
        return nsx;

    static const string NguyenAm = "aeiouAEIOU";//neu
gap cac nguyen am
    string result;//bo qua nguyen am
    for (char c : nsx)
    {
        unsigned char kyTu = static_cast<unsigned
char>(c);
        if (isalpha(kyTu) && NguyenAm.find(c) ==
string::npos)//ktra xem co phai chu cai, va cac chu
do co cac nguyen am khong
            result +=
static_cast<char>(toupper(kyTu));

        if (result.size() == 2)
            break;
    }
    return result;
}
```

#### 4.1.1 Nguyên lý thuật toán

- Hệ thống sử dụng một bảng băm tĩnh toàn cục **static map<string, int> DemMatHang** để lưu trữ số lượng sản phẩm hiện tại của từng thương hiệu.
- Khi nhập một sản phẩm, hệ thống trích xuất hoặc rút gọn tên nhà sản xuất thành một chuỗi tiền tố viết tắt (ví dụ: Xiaomi → XM, Samsung → SM).
- Tăng giá trị bộ đếm của tiền tố đó trong map: **DemMatHang[prefix]++**.
- Sử dụng thuật toán định dạng chuỗi để chuyển số đếm thành chuỗi 2 ký tự (nếu số thứ tự < 10, tự động chèn thêm ký tự '0' phía trước). Sau đó nối tiền tố với số thứ tự để ra mã cuối cùng (Ví dụ: XM + 01 → XM01).

```
string prefix = RutGonNSX(NSX);
DemMatHang[prefix]++; //dem sp của tung hang vi du
"[SN] = 1"
int stt = DemMatHang[prefix]; //so thu tu = so sp
SN01, SN02, ...
string chuoistt = (stt < 10) ? "0" +
to_string(stt) : to_string(stt); //them so "0" vao dang
sau neu stt < 10
maSP = prefix + chuoistt;
```

#### 4.1.2 Thuật toán phục hồi trạng thái bộ đếm từ File khi khởi động

Khi chương trình bật lên, nếu chỉ tăng từ 0, mã sản phẩm mới sẽ bị trùng với dữ liệu cũ trong file. Do đó, trong hàm **taoSanPhamTuDong()** tại **Menu.h**, hệ thống thực hiện giải thuật tách chuỗi (String Parsing): duyệt ngược mã sản phẩm từ cuối để tách phần số ra khỏi phần chữ, đổi phần số sang kiểu số nguyên bằng **stoi()**, sau đó dùng hàm **max()** để cập nhật lại giá trị lớn nhất cho **DemMatHang[tienTo]**. Thuật toán này giúp bộ đếm luôn tiếp tục chính xác từ vị trí cuối cùng của phiên làm việc trước.

```
size_t viTri = maSP.size();
```

```

        while (viTri > 0 && isdigit(static_cast<unsigned
char>(maSP[viTri - 1])))
            --viTri;

    if (viTri < maSP.size())
    {
        string tienTo = maSP.substr(0, viTri);
        int stt = stoi(maSP.substr(viTri));
        SanPham::DemMatHang[tienTo] =
max(SanPham::DemMatHang[tienTo], stt);
    }

```

## 4.2 Thuật toán Chuẩn hóa Dữ liệu Giá bán (Data Normalization)

Khi nhập giá bán, người dùng có thói quen nhập thêm các dấu chấm phân cách (ví dụ: 7.000.000) nhằm dễ đọc tránh nhầm số giá cả hoặc vô tình nhập sai ký tự (ví dụ định nhập 7.000.000 thì lại thành 7>)))>000 do loạn nút Shift, hoặc lại thành 70.00.00.00.0 do người dùng bị loạn dấu chấm).

Khi nhập dạng 7.000.000 chuẩn, nó là dạng char\*/string, dạng này dễ nhìn, trực quan cho con người, nhưng máy tính từ chối tính toán vì cơ bản bản chất nó là 1 xâu chuỗi ký tự, không phải các con số, do đó vẫn cần chuẩn hóa.

Tuy vậy, vì dạng string với dấu chấm rất dễ đọc với giá cả mệnh giá to nhiều số 0, vẫn cần tận dụng đến nó nhằm trực quan hóa cho con người sử dụng. Do đó vẫn sẽ tiến hành lưu trữ 1 bản string đánh dấu chấm chuẩn dựa trên giá bán đã được chuẩn hóa.

**Giải thuật xử lý:** Hàm **ChuanHoaGiaBan** nhận vào một chuỗi thô (**rawGiaBan**). Hàm sử dụng kiểu dữ liệu trả về dạng **pair <int,string>** nhằm return được **2 giá trị** sử dụng hậu tố first và second với kiểu return là int và string tương ứng. Nó thực hiện thuật toán duyệt qua từng ký tự của chuỗi: nếu ký tự đó là số (**isdigit**), nó sẽ giữ lại và nối vào một chuỗi kết quả mới; nếu là dấu chấm . hoặc ký tự lạ, nó sẽ bỏ

qua. Chuỗi số thuần túy thu được cuối cùng mới được chuyển đổi thành số nguyên kiểu **int** để lưu trữ vào biến **gia** và trả về **return thứ 1 ( .first() )** để ghi vào **GiaBan** của đối tượng, đảm bảo các phép tính nhân với số lượng mua sau này không bao giờ bị lỗi định dạng. Dạng chuẩn hóa đánh dấu chấm **string** của giá bán cũng được lưu lại vào biến **formatted** của hàm, khi cần sẽ dùng đến dưới dạng trả về **return thứ 2 ( .second() )**.

```
pair<int, string> SanPham::ChuanHoaGiaBan(const string
&rawGiaBan)
{
    string digits;
    for (char c : rawGiaBan)
    {
        if (isdigit((unsigned char)c))
            digits.push_back(c);
    }

    if (digits.empty())
        return {0, "0"};

    long long value = 0;
    for (char c : digits)
    {
        value = value * 10 + (c - '0');
        if (value > INT_MAX)
            break;
    }

    if (value > INT_MAX)
        value = INT_MAX;

    int gia = static_cast<int>(value);
```

```

    string formatted;
    string giaDaChuanHoa = to_string(gia);
    int count = 0;
    for (int i = (int)giaDaChuanHoa.size() - 1; i >=
0; --i)
    {
        formatted.insert(formatted.begin(),
giaDaChuanHoa[i]);
        if (++count == 3 && i > 0)
        {
            formatted.insert(formatted.begin(), '.');
            count = 0;
        }
    }

    return {gia, formatted};
}

```

Đoạn code sử dụng hàm thuật toán trên:

```

pair<int, string> parsedGia =
ChuanHoaGiaBan(giaBanRaw);
while (parsedGia.first <= 0)
{
    cout << "Gia ban khong hop le! Xin vui long
nhap lai: ";
    cin >> giaBanRaw;
    parsedGia = ChuanHoaGiaBan(giaBanRaw);
}

```

### 4.3 Thuật toán Đọc/Ghi và Phân tích Cú pháp Bản ghi phẳng (Flat-file Parsing) của SanPham

Vì hệ thống không sử dụng cơ sở dữ liệu công kênh, dữ liệu được lưu trữ trong tệp .txt theo kiến trúc tệp phẳng (Flat-file), sử dụng dấu gạch đứng (|) làm ký tự phân tách trường (Delimiter).

Giải thích thuật toán:

- Hàm **docTatCaSanPham** mở tệp dòng vào **ifstream** và đọc từng dòng một thông qua **getline(input, dong)** cho đến khi hết tệp.

```
vector<SanPham*> docTatCaSanPham(const string&
tenFile)
{
    vector<SanPham*> danhSach;
    ifstream input(tenFile);
    if (!input)
        return danhSach; // file không tồn tại hoặc
        không mở được

    string dong;
    while (getline(input, dong))
    {
        if (dong.empty())
            continue;

        if (!dong.empty() && dong.back() == '\\r')
            dong.pop_back();

        stringstream phanTach(dong);
```

```

        string loai, maSP, ten, nsx, giaBanStr,
soLuongStr;

        if (!getline(phanTach, loai, '|') ||
            !getline(phanTach, maSP, '|') ||
            !getline(phanTach, ten, '|') ||
            !getline(phanTach, nsx, '|') ||
            !getline(phanTach, giaBanStr, '|') ||
            !getline(phanTach, soLuongStr, '|'))
        {
            continue; // bo qua dong khong hop le
        }

        int giaBan = 0;
        int soLuong = 0;
        try
        {
            giaBan = stoi(giaBanStr);
            soLuong = stoi(soLuongStr);
        }
        catch (...)
        {
            continue;
        }

        SanPham* sanPham = taoSanPhamTuDong(loai,
maSP, ten, nsx, giaBan, soLuong);

        if (sanPham)
            danhSach.push_back(sanPham);
    }

```

```

    return danhSach;
}

```

- Hệ thống xử lý loại bỏ ký tự xuống dòng của hệ điều hành Windows (**\r**) nếu có bằng phương thức **dong.pop\_back()**.
- Chuyển đổi chuỗi dòng vừa đọc thành một luồng dữ liệu chuỗi **stringstream phanTach(dong)**
- Sử dụng vòng lặp **getline(phanTach, biến\_chứa, '|')** nối tiếp nhau để bóc tách chính xác từng trường dữ liệu theo thứ tự: *Loại sản phẩm, Mã sản phẩm, Tên, Nhà sản xuất, Giá bán, Số lượng* .
- Áp dụng khối lệnh **try - catch** để chuyển đổi chuỗi giá trị số (**GiaBanStr, SoLuongStr**) sang kiểu số nguyên **stoi** nhằm bắt mọi ngoại lệ dữ liệu tệp bị hỏng (Corrupted data). Nếu dòng hợp lệ, nó sẽ được chuyển tới hàm cấu trúc Factory Method để tạo thực thể tương ứng.

#### 4.4 Thuật toán Đọc/Ghi và Lưu trữ Hóa đơn

Trong kiến trúc của hệ thống, luồng dữ liệu hóa đơn được thiết kế theo dạng **Báo cáo định dạng có cấu trúc (Human-Readable Structured Report)** thay vì tệp phẳng phân tách bằng dấu gạch đứng ( | ) như phần **sản phẩm**. Điều này giúp tệp nhật ký hóa đơn hoạt động như một biên lai thực tế, dễ đọc đối với người quản lý.

##### 4.4.1 Thuật toán Lưu hóa đơn xuống file ( HoaDon::LuuFile )

Phương thức **LuuFile(const string &TenFile)** thực hiện nhiệm vụ chuyển đổi toàn bộ trạng thái của một đối tượng hóa đơn trên RAM (bao gồm mã hóa đơn, danh sách sản phẩm đã mua, số lượng, đơn giá) thành dữ liệu văn bản vĩnh viễn trên ổ cứng.

```

void HoaDon :: LuuFile(const string &TenFile) const
{

```



```

        ofstream outFile(TenFile, ios::app); //open file to
        continue write chu khong xoa di noi dung cu
        if (!outFile)
        {
            cerr << "Loi!Khong the mo file
"<<TenFile<<"!"<<endl;
            return;
        }
        outFile <<
"=====
====\n";
        outFile << " HOA DON BAN HANG: " << MaHD << "\n";
        outFile << "-----
-----\n";
        long long tongHoaDon = 0;
        for (size_t i = 0; i < DanhSachSP.size(); i++)
        {
            const ChiTietHoaDon &sp = DanhSachSP[i];
            int thanhTien = sp.giaBan * sp.soLuongMua;
            tongHoaDon += thanhTien;
            outFile << " "<<i+1<<". MaSP: "<<sp.maSP
                << " | Ten: " << sp.tenSP
                << " | NSX: " << sp.nsx
                << " | Don gia: " <<
SanPham::ChuanHoaGiaBan(to_string(sp.giaBan)).second
<< " VND"
                << " | SL: " << sp.soLuongMua
                << " | Thanh tien: " <<
SanPham::ChuanHoaGiaBan(to_string(thanhTien)).second
<< " VND\n";

```

```

    }

    outFile << "-----\n";

    outFile << " TONG TIEN: " <<
    SanPham::ChuanHoaGiaBan(to_string(tongHoaDon)).second
    << " VND\n";

    outFile <<
    "=====\n\n";

    outFile.close();

    cout << "Da luu hoa don "<<MaHD<<" vao file
    "<<TenFile<<endl;
}

```

- Cơ chế lưu trữ nối tiếp (Append Mode - ios::app):
  - **Bản chất giải thuật:** Thay vì sử dụng chế độ ghi đè mặc định (**ios::out**), thuật toán ép buộc luồng ra (**ofstream**) di chuyển con trỏ ghi xuống **cuối tệp** hiện tại.
  - **Ý nghĩa nghiệp vụ:** Cơ chế này đảm bảo tính toàn vẹn lịch sử giao dịch. Các hóa đơn cũ từ những phiên làm việc trước sẽ được giữ nguyên, hóa đơn mới sẽ xếp chồng nối tiếp xuống dưới, tạo thành một cuốn sổ cái nhật ký bán hàng liên tục.
- Giải thuật Duyệt tuyến tính và Định dạng dòng dữ liệu:
  - Thuật toán sử dụng một vòng lặp **for** chạy từ 0 đến ( **DanhSachSP.size() - 1** ) để quét qua vector chứa các mặt hàng khách hàng chọn mua.
  - Tại mỗi bước lặp, phần tử con trỏ SanPham\* hoặc cấu trúc ChiTietHoaDon được truy xuất để bóc tách thông tin định danh thông qua các hàm getter: getMaSP(), getNSX(), getGiaBan(). Dữ liệu được ghi vào file kèm các chuỗi ký tự phân cách trực quan như "=====" và "-----" kết hợp ký tự xuống

dòng \n. Điều này tạo cấu trúc phân cấp rõ ràng giữa tiêu đề hóa đơn, nội dung chi tiết các dòng sản phẩm, và tổng tiền thanh toán cuối cùng.

- Cơ chế đóng luồng an toàn (**outFile.close()**): Khi luồng ghi kết thúc, hàm `close()` được gọi để giải phóng tài nguyên hệ thống và kích hoạt lệnh đẩy toàn bộ dữ liệu còn sót lại trong bộ đệm RAM (Buffer Flush) xuống đĩa cứng một cách triệt để, ngăn chặn việc hỏng file dữ liệu.

#### 4.4.2 Thuật toán Đồng bộ Đọc/Ghi Hệ thống

Để chức năng lưu hóa đơn vận hành chính xác, hệ thống phải thực hiện một giải thuật phối hợp (gọi là **Luồng nghiệp vụ Giao dịch**) nằm trong hàm quản lý thực thi menu **lapHoaDonBanHang**. Luồng này liên kết chặt chẽ giữa việc **đọc file sản phẩm**, **trừ kho trên RAM**, **ghi file hóa đơn**, và **cập nhật lại file sản phẩm**.

```
void lapHoaDonBanHang(const string& tenFileSanPham,
const string& tenFileHoaDon)
{
    vector<SanPham*> danhSach =
docTatCaSanPham(tenFileSanPham);
    if (danhSach.empty())
    {
        cout << "Khong co san pham nao de ban.\n";
        return;
    }

    HoaDon hoaDon;
    string tenKhachHang;
    cout << "Nhap ten khach hang (bo trong neu la
khach le): ";
    getline(cin, tenKhachHang);
    hoaDon.setTenKH(tenKhachHang);
```

```

    bool daCoSanPham = false;
    while (true)
    {
        cout << "Nhap ma san pham can mua (2 de xoa
san pham gan nhat, 0 de ket thuc): ";
        string maSP;
        getline(cin, maSP);

        if (maSP == "0")
            break;

        if (maSP == "2")
        {
            -hoaDon;
            continue;
        }

        SanPham* sanPham = timSanPhamTheoMa(danhSach,
maSP);
        if (!sanPham)
        {
            cout << "Khong tim thay ma san pham: " <<
maSP << "\n";
            continue;
        }

        cout << "Thong tin san pham:\n";
        sanPham->Xuat();
    }

```

```

        if (hoaDon + sanPham)
            daCoSanPham = true;
    }

    if (!daCoSanPham)
    {
        cout << "Hoa don khong co san pham, khong luu
file.\n";
        giaiPhongDanhSachSanPham(danhSach);
        return;
    }

    cout << "\n";
    hoaDon.inHoaDon();
    hoaDon.LuuFile(tenFileHoaDon);

    if (luuDanhSachSanPham(danhSach, tenFileSanPham))
        cout << "Da cap nhat so luong ton kho trong
file " << tenFileSanPham << "\n";
    else
        cout << "Khong the cap nhat file san pham "
<< tenFileSanPham << "\n";

    giaiPhongDanhSachSanPham(danhSach);
}

```

Phân tích:

- Đọc cấu trúc kho hiện tại: Gọi hàm docTatCaSanPham() để nạp toàn bộ danh sách mặt hàng từ sanpham.txt vào một vector<SanPham\*> trên RAM.

- Khởi tạo đối tượng giao dịch: Tạo một thực thể HoaDon hd;, hệ thống tự động chạy constructor để sinh mã hóa đơn ngẫu nhiên MaHD bằng thuật toán rand().
- Vòng lặp mua sắm (Kiểm tra điều kiện biên tồn kho):
  - Người dùng nhập Mã SP cần mua. Hệ thống chạy **Thuật toán Tìm kiếm Tuyến tính** trên vector sản phẩm để định vị con trỏ đối tượng.
  - Người dùng nhập số lượng cần mua.
  - Kiểm tra điều kiện kho: Nếu **SoLuongMua > sp->getSoLuong()**, thuật toán chặn lại và đưa ra cảnh báo lỗi, không nạp vào hóa đơn.
  - Trừ kho tạm thời: Nếu kho đáp ứng đủ, hàm **sp->CheckSoLuongKho(SoLuongMua)** được kích hoạt để giảm số lượng tồn kho của chính đối tượng đó trên RAM. Đồng thời, nạp bản ghi này vào hóa đơn bằng toán tử nạp chồng hoặc hàm **themSanPham()**.
- Chốt phiên giao dịch và Lưu trữ đồng bộ song song:
  - Hệ thống gọi hd.LuuFile(TEN\_FILE\_HOADON) để ghi nhận lại biên lai giao dịch vừa thực hiện vào nhật ký hoaddon.txt ở chế độ ghi nối tiếp (ios::app).
  - Ngay sau đó, hệ thống gọi hàm ghiFileDanhSachSanPham() để duyệt lại toàn bộ vector<SanPham\*> trên RAM (lúc này đã bị trừ bớt số lượng do có hàng được bán) và thực hiện **ghi đè hoàn toàn** (ios::out) lên tệp sanpham.txt.

Giải thuật phối hợp này đảm bảo tính nhất quán dữ liệu (Data Consistency). Số lượng hàng hóa trong kho thực tế (sanpham.txt) luôn giảm đi một lượng hoàn toàn khớp với tổng số lượng sản phẩm được ghi nhận đã bán ra trong nhật ký hóa đơn (hoaddon.txt).

#### 4.4.3 Thuật toán Đọc và Xử lý nâng cao dữ liệu Hóa đơn

Trong file Menu.h, nhóm cũng đã bổ sung các hàm xử lý luồng đọc file hóa đơn phức tạp hơn như hiển thị hóa đơn mới nhất hoặc xóa hóa đơn cũ nhất

- Giải thuật hiển thị hóa đơn mới nhất (inHoaDonMoiNhatTuFile):

```
void inHoaDonMoiNhatTuFile(const string& tenFile)
{
    vector<string> danhSachHoaDon =
docTatCaHoaDon(tenFile);
    if (danhSachHoaDon.empty())
    {
        cout << "Chua co hoa don nao trong file " <<
tenFile << "\n";
        return;
    }

    cout << "=== HOA DON MOI NHAT ===\n";
    cout << danhSachHoaDon.back();
}
```

- Vì file hóa đơn ghi nối tiếp, hóa đơn mới nhất luôn nằm ở **cuối tệp**.
- Thuật toán tiến hành mở file đầu vào ifstream. Thay vì xử lý từ đầu đến cuối tồn bộ nhớ, thuật toán sử dụng một mảng lưu trữ tạm thời các khối dòng, hoặc đọc toàn bộ các khối ngăn cách bởi dấu "=====".
- Khối hóa đơn cuối cùng được tách ra thông qua bộ lọc ký tự tiêu đề và in ngược lại ra console để hiển thị cho người dùng.

- Giải thuật Xóa hóa đơn cũ nhất (xoaHoaDonCuNhat):

```
bool xoaHoaDonCuNhat(const string& tenFile)
{
    vector<string> danhSachHoaDon =
docTatCaHoaDon(tenFile);
    if (danhSachHoaDon.empty())
        return false;

    danhSachHoaDon.erase(danhSachHoaDon.begin());

    ofstream output(tenFile, ios::trunc);
    if (!output)
        return false;

    for (size_t i = 0; i < danhSachHoaDon.size();
++i)
    {
        output << danhSachHoaDon[i];
        if (i + 1 < danhSachHoaDon.size())
            output << "\n";
    }

    return true;
}
```



- Áp dụng mô hình **Hàng đợi tệp (File Queue)**. Hóa đơn cũ nhất nằm ở **đầu tệp**.
- Thuật toán đọc toàn bộ nội dung file hoaddon.txt vào bộ nhớ, xác định vị trí kết thúc của khối hóa đơn đầu tiên (dựa vào dấu phân cách =====).
- Bỏ qua khối đầu tiên này, thuật toán lấy toàn bộ phần nội dung còn lại của các hóa đơn phía sau và ghi đè ngược trở lại file hoaddon.txt, thực hiện thành công thao tác xóa bản ghi cũ nhất khỏi hệ thống.

## 5 Kết quả thực nghiệm – Demo chương trình

### 5.1 Môi trường thực nghiệm

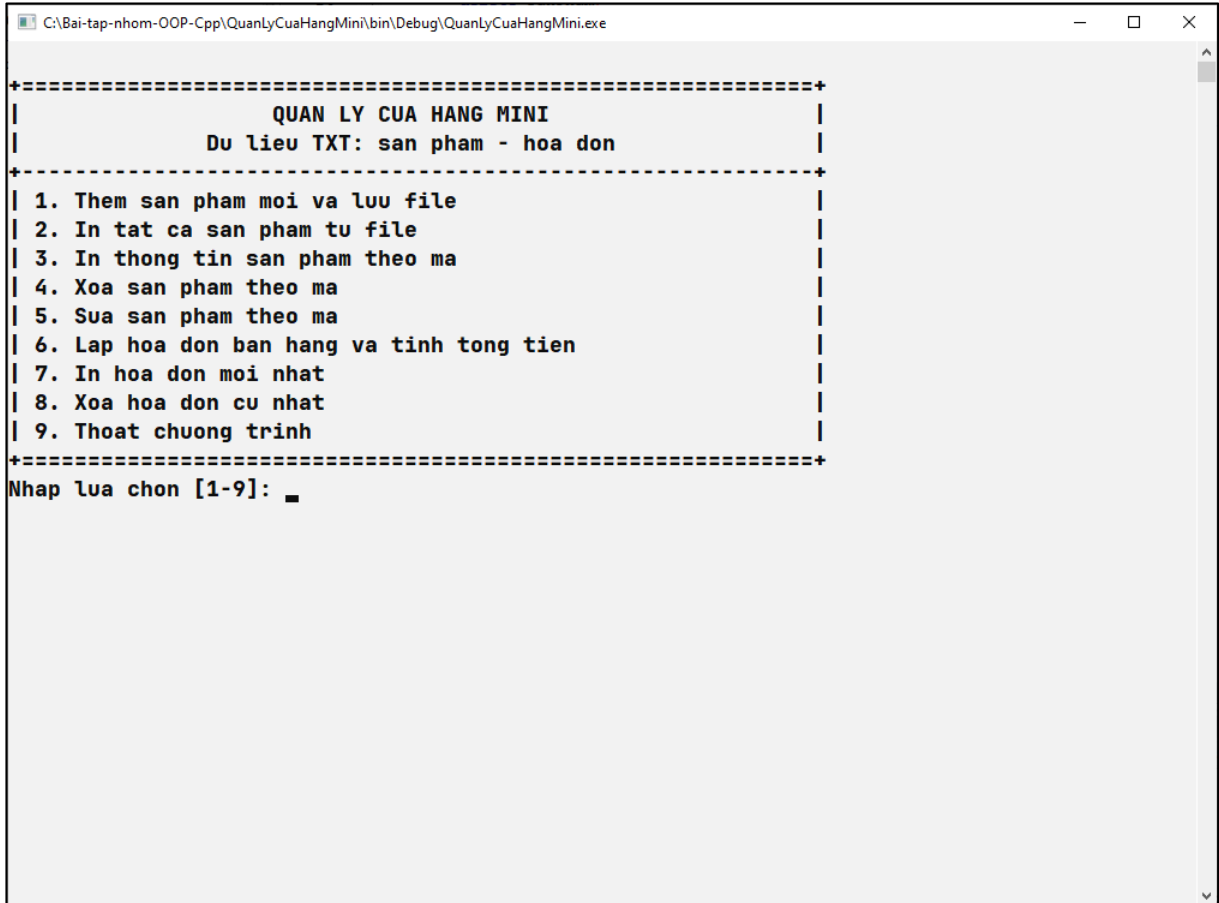
- Trình biên dịch & IDE: GCC Compiler + Code::Blocks
- NLTT: C++ chuẩn C++ 11 trở lên.
- Giao diện: Console Application (Menu tương tác nhập số)
- CSDL: Hệ thống tệp phẳng (Flat-file storage) lưu trữ qua sanpham.txt và hoaddon.txt.

### 5.2 Kịch bản kiểm thử và Demo chức năng hệ thống

Có nhiều tính năng nhưng nhóm sẽ trình bày kịch bản của tính năng hay được sử dụng nhiều nhất hàng ngày (bởi người dùng/nhân viên thu ngân): Xem mọi thông tin sản phẩm và Lập hóa đơn

### 5.2.1 Giao diện Menu chính khi khởi động chương trình

Khi chương trình được kích hoạt, hệ thống tự động nạp dữ liệu sản phẩm từ file **sanpham.txt** lên RAM và hiển thị Menu điều khiển:

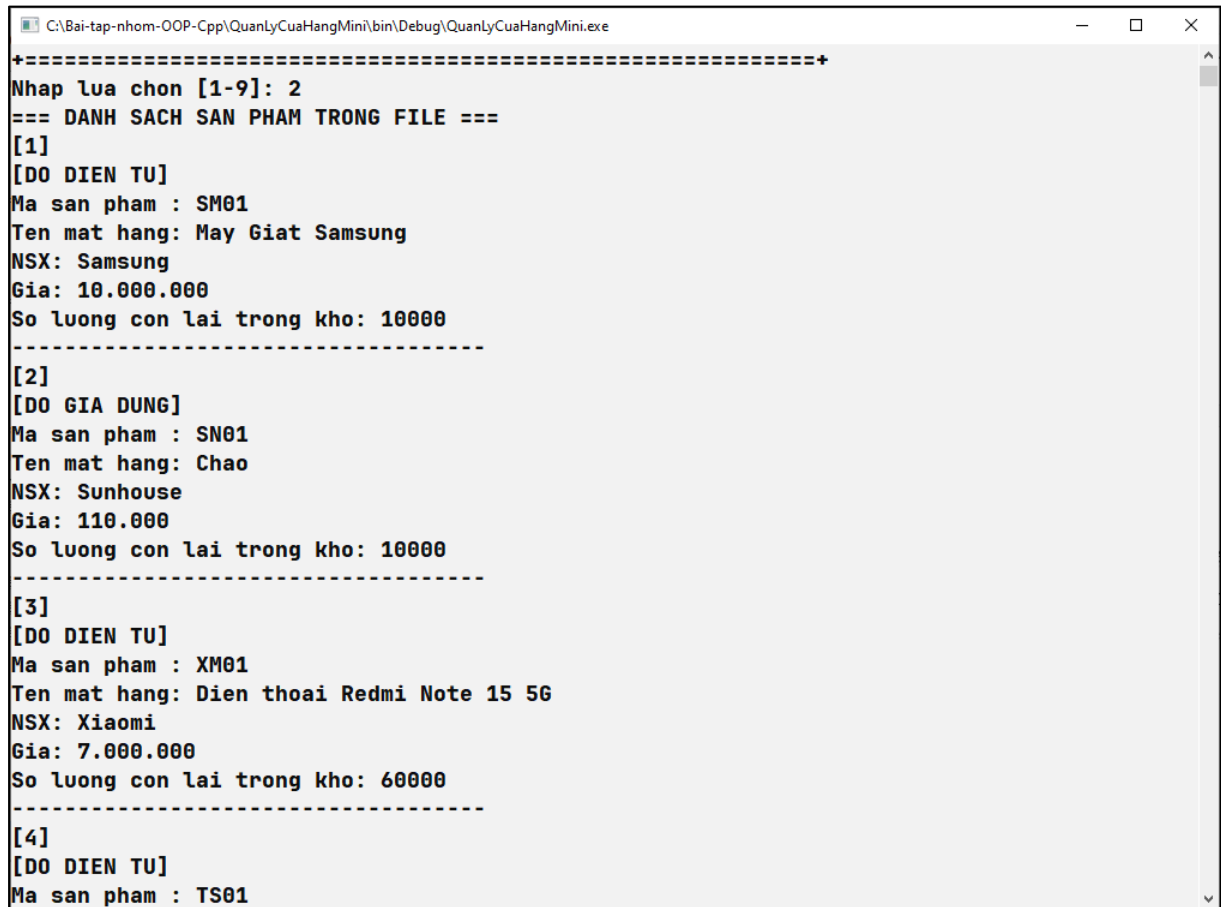


```
C:\Bai-tap-nhom-OOP-Cpp\QuanLyCuaHangMini\bin\Debug\QuanLyCuaHangMini.exe
+=====+
|          QUAN LY CUA HANG MINI          |
|          Du lieu TXT: san pham - hoa don          |
+-----+
| 1. Them san pham moi va luu file           |
| 2. In tat ca san pham tu file              |
| 3. In thong tin san pham theo ma           |
| 4. Xoa san pham theo ma                    |
| 5. Sua san pham theo ma                    |
| 6. Lap hoa don ban hang va tinh tong tien  |
| 7. In hoa don moi nhat                     |
| 8. Xoa hoa don cu nhat                     |
| 9. Thoat chuong trinh                     |
+-----+
Nhap lua chon [1-9]: _
```

### 5.2.2 Chức năng Hiển thị danh sách và Chuẩn hóa dữ liệu (Chức năng 2)

Khi chọn chức năng 2, hệ thống đọc dữ liệu đã phân loại để hiển thị ra màn hình.

Minh họa dữ liệu đầu ra được căn lề thẩm mỹ:



```
+=====+
Nhap lua chon [1-9]: 2
=== DANH SACH SAN PHAM TRONG FILE ===
[1]
[DO DIEN TU]
Ma san pham : SM01
Ten mat hang: May Giat Samsung
NSX: Samsung
Gia: 10.000.000
So luong con lai trong kho: 10000
-----
[2]
[DO GIA DUNG]
Ma san pham : SN01
Ten mat hang: Chao
NSX: Sunhouse
Gia: 110.000
So luong con lai trong kho: 10000
-----
[3]
[DO DIEN TU]
Ma san pham : XM01
Ten mat hang: Dien thoai Redmi Note 15 5G
NSX: Xiaomi
Gia: 7.000.000
So luong con lai trong kho: 60000
-----
[4]
[DO DIEN TU]
Ma san pham : TS01
```

### 5.2.3 Chức năng Lập hóa đơn và Đồng bộ hóa kho hàng (Chức năng 6)

Khi người dùng chọn chức năng 6, hệ thống cho phép tạo phiên giao dịch mới:

**Bước 1:** Nhập tên khách hàng.

**Bước 2:** Nhập mã sản phẩm và số lượng mua cần giao dịch.

**Bước 3:** Hệ thống tự động kiểm tra tồn kho (Biên điều kiện). Nếu hợp lệ, tiến hành trừ kho tạm thời trên RAM và đẩy vào hóa đơn qua toán tử nạp chồng +.

*Ví dụ tiến trình tương tác nhập liệu:*

- Tên khách hàng: Nguyễn Hồng Nam
- Mã sản phẩm cần mua: XM01, SM01
- Số lượng: 01, 01

```
C:\Bai-tap-nhom-OOP-Cpp\QuanLyCuaHangMini\bin\Debug\QuanLyCuaHangMini.exe

+=====+
|          QUAN LY CUA HANG MINI          |
|          Du lieu TXT: san pham - hoa don          |
+-----+
| 1. Them san pham moi va luu file          |
| 2. In tat ca san pham tu file            |
| 3. In thong tin san pham theo ma         |
| 4. Xoa san pham theo ma                  |
| 5. Sua san pham theo ma                  |
| 6. Lap hoa don ban hang va tinh tong tien |
| 7. In hoa don moi nhat                    |
| 8. Xoa hoa don cu nhat                    |
| 9. Thoat chuong trinh                    |
+-----+
Nhap lua chon [1-9]: 6
Nhap ten khach hang (bo trong neu la khach le): Nguyen Hong Nam
Nhap ma san pham can mua (2 de xoa san pham gan nhat, 0 de ket thuc): XM01
Thong tin san pham:
[DO DIEN TU]
Ma san pham : XM01
Ten mat hang: Dien thoai Redmi Note 15 5G
NSX: Xiaomi
Gia: 7.000.000
So luong con lai trong kho: 60000Nhap so luong mua: 1
Da them [XM01] x1 vao hoa don Ma HD 1041
Nhap ma san pham can mua (2 de xoa san pham gan nhat, 0 de ket thuc):
```

```
C:\Bai-tap-nhom-OOP-Cpp\QuanLyCuaHangMini\bin\Debug\QuanLyCuaHangMini.exe

| 2. In tat ca san pham tu file          |
| 3. In thong tin san pham theo ma         |
| 4. Xoa san pham theo ma                  |
| 5. Sua san pham theo ma                  |
| 6. Lap hoa don ban hang va tinh tong tien |
| 7. In hoa don moi nhat                    |
| 8. Xoa hoa don cu nhat                    |
| 9. Thoat chuong trinh                    |
+-----+
Nhap lua chon [1-9]: 6
Nhap ten khach hang (bo trong neu la khach le): Nguyen Hong Nam
Nhap ma san pham can mua (2 de xoa san pham gan nhat, 0 de ket thuc): XM01
Thong tin san pham:
[DO DIEN TU]
Ma san pham : XM01
Ten mat hang: Dien thoai Redmi Note 15 5G
NSX: Xiaomi
Gia: 7.000.000
So luong con lai trong kho: 60000Nhap so luong mua: 1
Da them [XM01] x1 vao hoa don Ma HD 1041
Nhap ma san pham can mua (2 de xoa san pham gan nhat, 0 de ket thuc): SM01
Thong tin san pham:
[DO DIEN TU]
Ma san pham : SM01
Ten mat hang: May Giat Samsung
NSX: Samsung
Gia: 10.000.000
So luong con lai trong kho: 10000Nhap so luong mua: 1
Da them [SM01] x1 vao hoa don Ma HD 1041
Nhap ma san pham can mua (2 de xoa san pham gan nhat, 0 de ket thuc):
```

Ngay sau khi hoàn thành giao dịch, chương trình tự động xuất ra mẫu biên lai in lên console đồng thời cập nhật tức thời xuống file lưu trữ hoadon.txt:

```
C:\Bai-tap-nhom-OOP-Cpp\QuanLyCuaHangMini\bin\Debug\QuanLyCuaHangMini.exe
Da them [SM01] x1 vao hoa don Ma HD 1041
Nhap ma san pham can mua (2 de xoa san pham gan nhat, 0 de ket thuc): 0

====HOA DON MAT HANG====
Ma HD: Ma HD 1041
Ten KH: Nguyen Hong Nam
1. Ma SP: XM01 | Ten: Dien thoai Redmi Note 15 5G | NSX: Xiaomi | Don gia: 7.000.000 VND |
SL: 1 | Thanh tien: 7.000.000 VND
2. Ma SP: SM01 | Ten: May Giat Samsung | NSX: Samsung | Don gia: 10.000.000 VND | SL: 1 |
Thanh tien: 10.000.000 VND

TONG TIEN THANH TOAN: 17.000.000 VND
Da luu hoa don Ma HD 1041 vao file hoadon.txt
Da cap nhat so luong ton kho trong file sanpham.txt

+=====+
|          QUAN LY CUA HANG MINI          |
|          Du lieu TXT: san pham - hoa don          |
+-----+
| 1. Them san pham moi va luu file          |
| 2. In tat ca san pham tu file            |
| 3. In thong tin san pham theo ma        |
| 4. Xoa san pham theo ma                  |
| 5. Sua san pham theo ma                  |
| 6. Lap hoa don ban hang va tinh tong tien |
| 7. In hoa don moi nhat                    |
| 8. Xoa hoa don cu nhat                    |
| 9. Thoat chương trình                      |
+-----+
Nhap lua chon [1-9]:
```

Đồng thời, khi kiểm tra lại file dữ liệu gốc sanpham.txt, số lượng của mã SM01 từ 10000 đã được thuật toán đồng bộ tự động hạ xuống chính xác còn 9999 trong bản ghi.

## 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

### 1. Kết quả đạt được

- **Tính đúng đắn và nghiệp vụ:** Hệ thống đã hoàn thiện đầy đủ các tính năng cốt lõi theo đúng yêu cầu bài toán quản lý bán hàng thực tế bao gồm: thêm, sửa, xóa, tìm kiếm sản phẩm và tạo lập hóa đơn, trừ kho tự động.

- **Ứng dụng OOP vững chắc:** Mã nguồn áp dụng thành công các nguyên lý cơ bản của Lập trình hướng đối tượng:

- **Tính đóng gói:** Bảo vệ toàn vẹn các thuộc tính của sản phẩm và hóa đơn thông qua từ khóa `private` và các bộ hàm `Getter/Setter`.
- **Tính kế thừa & Đa hình:** Phân tách rõ ràng lớp cha **SanPham** và các lớp con chuyên biệt **DoDienTu**, **DoGiaDung**. Sử dụng cơ chế ghi đè phương thức hàm ảo (**virtual, override**) và hàm hủy ảo, triệt tiêu hoàn toàn lỗi rò rỉ bộ nhớ (Memory Leak).
- **Nạp chồng toán tử:** Thiết kế thành công nạp chồng toán tử cộng (+) để thêm hàng và toán tử trừ (-) để hoàn tác (Undo), giúp mã nguồn tối giản và sáng rõ về mặt thuật toán.

- **Xử lý dữ liệu tối ưu:** Xây dựng thành công bộ lọc chuẩn hóa chuỗi dữ liệu đầu vào (loại bỏ ký tự phân cách nghìn khi nhập giá) và giải thuật đồng bộ dữ liệu RAM-Disk hai chiều giúp hệ thống không bị mất dữ liệu khi tắt máy đột ngột.

### 2. Hướng phát triển

- **Nâng cấp giao diện đồ họa (GUI):** Chuyển đổi mã nguồn từ Console truyền thống sang giao diện ứng dụng Desktop hoàn chỉnh bằng cách tích hợp các thư viện đồ họa mạnh mẽ của C++ như Qt Framework, SFML hoặc wxWidgets.
- **Tích hợp Hệ quản trị cơ sở dữ liệu chuyên dụng:** Thay thế hoàn toàn hệ thống file văn bản phẳng .txt bằng các hệ quản trị cơ sở dữ liệu quan hệ gọn nhẹ như **SQLite** (nhúng trực tiếp vào ứng dụng) hoặc **MySQL / PostgreSQL** để tối ưu hóa tốc độ truy vấn bằng ngôn ngữ SQL, tăng cường tính toàn vẹn và bảo mật dữ liệu.

- **Phát triển tính năng bảo mật:** Xây dựng thêm lớp đối tượng TaiKhoan (User Authentication) nhằm thực hiện tính năng Đăng nhập, phân quyền thao tác chi tiết (Admin được sửa/xóa kho hàng, Nhân viên chỉ được quyền lập hóa đơn).
- **Mở rộng tính năng Thống kê chuyên sâu:** Ứng dụng các thuật toán sắp xếp và gom nhóm để xây dựng bộ lọc báo cáo doanh thu chi tiết theo mốc thời gian (ngày/tháng/năm) hoặc theo danh mục nhóm ngành hàng ăn khách nhất dưới dạng biểu đồ trực quan.

## 7 DANH MỤC TÀI LIỆU THAM KHẢO

- [1] ThS. Ninh Thị Thanh Tâm, *Giáo trình và Slide Bài giảng Lập trình hướng đối tượng với C++*, Khoa Công nghệ thông tin, Học viện Quản lý Giáo dục, 2026.
- [2] Robert Lafore, *Object-Oriented Programming in C++*, 4th Edition, Sams Publishing, 2002.
- [3] Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley, 2013.
- [4] Walter Savitch, *Absolute C++*, 6th Edition, Pearson, 2015.
- [5] W3Schools Online Web Tutorials, *C++ Tutorial & OOPs Concepts*, Available at: <https://www.w3schools.com/cpp/>
- [6] Cppreference Official Documentation, *Standard Technology Library (STL) & Input/Output File Streams (std::ofstream, std::ifstream)*, Available at: <https://en.cppreference.com/>.
- [7] GeeksforGeeks Nền tảng Khoa học Máy tính, *Object-Oriented Programming in C++ (Inheritance, Polymorphism & Virtual Functions)*, Available at: <https://www.geeksforgeeks.org/cpp-oops-concepts/>.
- [8] Google Gemini (Large Language Model), *Hỗ trợ phân tích giải thuật đọc/ghi file đồng bộ, tối ưu hóa thuật toán tìm kiếm tuyến tính mã sản phẩm và định dạng cấu trúc in hóa đơn thẩm mỹ dạng bảng*, Google AI, Hệ thống tương tác dữ liệu phiên bản 2026.
- [9] Anthropic Claude (Claude 3.5 Sonnet / Claude 3 Opus), *Trợ lý tư vấn giải pháp xử lý bộ lọc loại bỏ dấu chấm phân cách hàng nghìn khi ép kiểu dữ liệu tiền tệ (string sang int) và xây dựng cơ chế hàm hủy ảo (Virtual Destructor) triệt tiêu lỗi rò rỉ bộ nhớ (Memory Leak)*, Anthropic PBC, 2026.
- [10] OpenAI Codex / ChatGPT, *Hỗ trợ cấu trúc cú pháp nâng cao cho nạp chồng toán tử cộng (+) thêm sản phẩm và toán tử trừ (-) hoàn tác (Undo) danh sách liên kết lưu trữ tạm thời*, OpenAI, 2026.



[11] Google NotebookLM (AI Research Assistant), *Công cụ hỗ trợ tổng hợp, hệ thống hóa toàn bộ các file mã nguồn (.h, .cpp) kết hợp tài liệu đặc tả và hướng dẫn chấm bài tập lớn nhằm xây dựng bố cục báo cáo chuẩn hóa quy chuẩn*, Google Labs, 2026.

**8 PHÂN CÔNG CÔNG VIỆC NHÓM**

| <b>Thành viên</b> | <b>Công việc được phân công</b> | <b>Tỉ lệ đóng góp</b> |
|-------------------|---------------------------------|-----------------------|
| Họ và tên SV1     | Nguyễn Minh Đức                 | 40                    |
| Họ và tên SV2     | Nguyễn Hồng Nam                 | 40                    |
| Họ và tên SV3     | Nguyễn Văn Huy                  | 20                    |